Field Hours

CSCE 620 Computational Geometry

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Turns and the cost of a test

You test the turn from A to B to C with the cross product (Bx - Ax) times (Cy - Ay) minus (By - Ay) times (Cx - Ax). Place A at (0, 0), B at (4, 0), and C at (1, 2). The value equals 8, so the path turns left. A negative value is a right turn, and zero means collinear. Later algorithms call this test and do not add a second geometric primitive.

You also name the cost model before you pick an algorithm. A test on coordinates that fit in a machine word is one step. Sorting n points by x-coordinate costs on the order of n log n tests, and for n = 8 that is about 24 comparisons. A hull or a triangulation that hides an extra quadratic pass loses to one that sorts once and then scans. You count tests in this model.

Sign of one cross product

Positive 8

Path A, B, C turns left

C sits above segment AB

Negative sign

Path A, B, C turns right

C would sit below AB

Left turn at height 2

1. A is (0, 0), B is (4, 0), and C is (1, 2).

2. The cross product is 4 times 2 minus 0 times 1, which equals 8.

3. The sign is positive, so you record a left turn. The point C at (1, -2) would give -8, a right turn.

Problem 1. A is (0, 0), B is (2, 0), and C is (2, 5). What is the cross product, and which way does the path turn?

Answer. It equals 2 times 5 minus 0 times 2, which is 10. The turn is left.

Problem 2. A is (1, 1), B is (3, 1), and C is (5, 1). What does the test return, and what do you conclude?

Answer. It equals 2 times 0 minus 0 times 4, which is 0. The three points are collinear.

Lecture 2 · 90 minutes

Convex hull by a monotone chain

You build a convex hull with Andrew's monotone chain. Sort the points by increasing x-coordinate, breaking ties by y, then scan left to right. Keep a lower chain that makes only left turns. When the newest point makes a right turn or a flat turn with the last two points, pop the middle one and test again. On 5 points with 1 inside, that interior point is popped and the hull keeps the other 4.

You pay for the sort, not for the pops. Each point is pushed once and popped at most once, so the lower scan does at most n pops. With n = 5 that is at most 5 pops below and 5 on the upper chain, which scans right to left with the same left-turn test. You glue the chains at the two extreme points. The n log n term is the sort.

Lower chain, left to right

1. Sort by x, then by y

2. Append the next point

3. Pop any right turn

4. Keep the left turns

Five points, one inside

1. Sorted order is (0, 0), (0, 3), (2, 1), (4, 0), (4, 3).

2. The lower scan pops (2, 1) because (0, 0), (2, 1), (4, 0) is a right turn. The lower chain ends at (0, 0), (4, 0), (4, 3).

3. The upper scan, right to left, also pops (2, 1). After dropping duplicated ends the hull is the four corners.

Problem 1. On (0, 0), (1, 1), (2, 0), does the lower scan keep (1, 1)? Is that point still a hull vertex?

Answer. No. The cross product of that triple is -2, a right turn, so the lower scan pops (1, 1). The upper scan keeps it. All 3 points are hull vertices.

Problem 2. A lower scan runs on n = 9 points. At most how many pops can it perform, and which term still dominates the whole hull algorithm?

Answer. At most 9 pops, one per point. The scan is linear. The sort, on the order of n log n comparisons, still dominates.

Lecture 3 · 90 minutes

Crossings by a vertical sweep

You find segment crossings by sweeping a vertical line from left to right. The status stores the segments that meet the sweep line, ordered by the y where they cross it. Two segments can begin an intersection only after they become neighbors in that order, so you test neighbors rather than every pair. With 4 segments the naive pair count is 6. The sweep tests a pair only when the status makes them adjacent.

You handle endpoints as events. A left endpoint inserts a segment and creates at most 2 new neighbor pairs. A

right endpoint deletes a segment. For n segments and k crossings, a balanced status tree costs about (n + k) log n steps. Take n = 4 and k = 1. You still report that single crossing, and you do not pay a quadratic 6 for every input of this size when some segments never become neighbors.

Report one neighbor crossing

Insert the left ends -> Test the new neighbors -> Queue the crossing -> Swap their order

Two segments cross at (2, 2)

1. Segment P runs from (0, 0) to (4, 4), so y = x. Segment Q runs from (0, 4) to (4, 0), so y = 4 - x.

2. Both left endpoints lie at x = 0, so the two segments are neighbors and you are allowed to test that pair.

3. Set x = 4 - x. Then x = 2 and y = 2. The sweep reports (2, 2) and swaps the order just to the right of it.

Problem 1. Just to the right of x = 2, which of P and Q is above the other?

Answer. P is above Q. For x greater than 2, y = x exceeds y = 4 - x. The sweep swaps them at the crossing.

Problem 2. Five segments give how many naive pairs? Why can the sweep test fewer than that?

Answer. 5 times 4 over 2 equals 10 naive pairs. The sweep tests a pair only when the two segments are neighbors on the sweep line, so separated segments are never paired.

Lecture 4 · 90 minutes

Ears and a triangulation count

You triangulate a simple polygon with n vertices into n - 2 triangles, using n - 3 diagonals. One diagonal clips a triangle and leaves a polygon on n - 1 vertices, which needs (n - 1) - 2 = n - 3 triangles, and adding the clipped triangle gives n - 2. For a hexagon, n = 6, so you draw 3 diagonals and obtain 4 triangles. The count does not depend on which legal diagonal you draw first.

You find a legal diagonal by clipping an ear. An ear is three consecutive vertices a, b, c such that segment a-c lies inside the polygon and no vertex lies inside triangle abc. A simple polygon with n greater than 3 has at least 2 ears, so you do not get stuck. A convex hexagon has 6 ears. One reflex vertex leaves fewer ears, and the lower bound of 2 still holds.

Clipping a hexagon

Start with 6 vertices

Clip one ear, 5 left

Clip again, 4 left

Finish with 4 triangles

A convex quadrilateral

1. A convex quadrilateral has n = 4, so the formulas give n - 2 = 2 triangles and n - 3 = 1 diagonal.

2. Either diagonal lies inside, and every vertex is an ear. Clip one vertex by drawing that single diagonal.

3. The remainder is already a triangle, so you stop. Two triangles and one diagonal match the formulas.

Problem 1. A simple polygon has n = 10 vertices. How many triangles, and how many diagonals, are in any triangulation?

Answer. Triangles: 10 - 2 = 8. Diagonals: 10 - 3 = 7.

Problem 2. You clip one ear from a polygon of 7 vertices. How many vertices remain, and how many triangles does that remaining polygon need?

Answer. 6 vertices remain. That polygon needs 6 - 2 = 4 triangles. With the clipped ear, the original total is 5, matching 7 - 2.

Lecture 5 · 90 minutes

Voronoi cells from bisectors

You define the Voronoi cell of a site as the points closer to that site than to any other. For 2 sites the boundary is the perpendicular bisector, and you get 2 unbounded cells. For 3 sites not on one line, the bisectors meet at the center of the circle through the sites. You compute that one center. Each cell is a convex region cut out by bisectors.

You check the center on sites (0, 0), (6, 0), and (0, 6). A right triangle has its circumcenter at the midpoint of the hypotenuse, here (3, 3). The squared distance from (3, 3) to each site is 18. A query at (1, 1) has squared distance 2 to (0, 0) and 26 to each other site. The cell of the origin owns (1, 1).

Two sites, then three

Two sites

One bisector line

Two unbounded cells

Three sites

Bisectors meet once

Vertex at (3, 3)

Right triangle, center (3, 3)

1. Sites A (0, 0), B (6, 0), and C (0, 6). The midpoint of B and C is (3, 3).

2. Squared distance from (3, 3) to A is 9 + 9 = 18, and the distances to B and to C match it.

3. The Voronoi vertex is (3, 3). The query (1, 1) has squared distances 2, 26, and 26, so A's cell owns it.

Problem 1. Which site owns (5, 1)? Compare squared distances to A (0, 0), B (6, 0), and C (0, 6).

Answer. Squared distances are 26 to A, 2 to B, and 50 to C. B owns (5, 1).

Problem 2. A point is equidistant from A and B and closer to both than to C. Why is it not interior to only one cell?

Answer. Equal distance to A and B puts it on their shared boundary. C does not own it. Boundary points are not interior to a single cell.

Lecture 6 · 90 minutes

Empty circles and nearest sites

You draw a Delaunay edge when two Voronoi cells share a boundary. The test you run is the empty circle: keep a triangle when the circle through its vertices holds no other site. If a fourth site lies inside, you flip. On sites (0, 0), (6, 0), and (0, 6) there is no fourth site, so that one triangle is Delaunay and all 3 edges stay.

You find a nearest neighbor among the Delaunay neighbors, not among all n - 1 other sites. In a convex quadrilateral, n = 4, one diagonal leaves two vertices of degree 2. Take the rectangle (0, 0), (4, 0), (4, 3), (0, 3) with diagonal (0, 0) to (4, 3). Vertex (4, 0) then has only 2 neighbors, (0, 0) and (4, 3). You compare those 2 distances, not 3.

Degree after one diagonal

Four corners of a rectangle

One diagonal is drawn

Two vertices have degree 2

Compare 2 distances, not 3

Nearest neighbor of (4, 0)

1. With diagonal (0, 0) to (4, 3), the neighbors of (4, 0) are (0, 0) and (4, 3).

2. The distance to (4, 3) is 3. The distance to (0, 0) is 4. The non-neighbor (0, 3) is 5 away.

3. The nearest site is (4, 3), at distance 3, and it is one of the 2 Delaunay neighbors.

Problem 1. A site in a 12-point set has Delaunay degree 3. How many distances do you compute to find its nearest neighbor, against a full scan?

Answer. You compute 3 distances, not 11. The nearest neighbor is one of the Delaunay neighbors.

Problem 2. Point D lies strictly inside the circle through triangle ABC. Is ABC a Delaunay triangle?

Answer. No. The empty-circle test fails. You flip or rebuild so that every circumcircle is empty.

Lecture 7 · 90 minutes

Rectangles on a kd-tree

You store points in a kd-tree by alternating split axes. The root splits on x, the children split on y, and the next level splits on x again. Each node owns an axis-aligned region. A query rectangle visits a node only when the regions meet, and it reports a point only when the point is inside. In 2D the cost is on the order of square root of n plus k hits.

You can see a prune on 4 points: (1, 1), (2, 5), (6, 2), and (7, 6). The root splits at x = 6, left holding the first pair and right the second. The query x from 0 to 3 and y from 0 to 3 ends before the split. You drop the right child, skipping 2 points. The left side reports only (1, 1), so k = 1.

Prune at the root split

Root cuts at x = 6

Left holds two points

Right child is dropped

Report k = 1

Query that misses the right half

1. The root split at x = 6 puts (1, 1) and (2, 5) on the left and (6, 2) and (7, 6) on the right.

2. The query rectangle stops at x = 3, so it cannot meet the right region. You skip 2 points in one test.

3. On the left, (1, 1) lies inside x 0 to 3 and y 0 to 3. The point (2, 5) does not. You report k = 1.

Problem 1. Same tree, but the query is x from 0 to 3 and y from 4 to 6. What is k, and which point is it?

Answer. k = 1, the point (2, 5). The right child is still pruned. (1, 1) has y = 1, outside 4 to 6.

Problem 2. The 2D range bound is on the order of square root of n plus k. For n = 100 and k = 3, what number is that?

Answer. Square root of 100 is 10, plus k = 3, which is 13. That is an order-of-cost expression, not a claim that every query touches exactly 13 nodes.

Lecture 8 · 90 minutes

What you can now build

You can now build a convex hull, a Voronoi diagram with its Delaunay dual, and a kd-tree answer to a rectangle. Each one uses the orientation or distance tests you started with, including the cross product that equaled 8. For n = 8 a sort is about 24 comparisons. A triangulation with h hull vertices has 2n - 2 - h triangles. You name h before you use that formula.

You keep hull answers and query answers apart. For n = 8 and h = 5 the formula gives 16 - 2 - 5 = 9 triangles, and the interior count is 8 - 5 = 3. A range query with k = 1 did not find those 3 interior points. It found the points inside one rectangle. You can build the objects, and you can say which number came from which question.

Hull count versus a query

Hull, n = 8

h = 5 extreme points

9 triangles in the set

One range query

Rectangle, not the hull

Reports k = 1 only

Eight points, five on the hull

1. n = 8 and h = 5. The triangle count is 2 times 8 minus 2 minus 5.

2. That is 16 - 2 - 5 = 9 triangles. Interior points are 8 - 5 = 3.

3. A query with k = 1 answers a different question. The value 1 is not the interior count 3.

Problem 1. A set has n = 10 and h = 6. How many triangles does a triangulation have, and how many points are interior?

Answer. Triangles: 2 times 10 minus 2 minus 6 = 12. Interior points: 10 - 6 = 4.

Problem 2. Combine the hull formula with a range count. For n = 8 and h = 5, how many triangles are there, and does a query result k = 1 equal the number of interior points?

Answer. Triangles: 16 - 2 - 5 = 9. Interior points: 3. The query result 1 is not 3. A rectangle count and a hull count are different questions.